Enterprise AI Platform — Architecture & Engineering Handbook

Version: 1.0
Date: June 2026
Status: Published
Audience: Senior Engineers, Staff Engineers, Principal Engineers, Architects, AI Platform Engineers

---

Table of Contents

1. Executive Summary
2. Platform Vision
3. Design Philosophy
4. System Context
5. Module Overview
6. Runtime Architecture
7. Document Lifecycle
8. AI Architecture
9. Semantic Enrichment
10. GraphRAG
11. Retrieval Architecture
12. Prompt Architecture
13. Model Architecture
14. Workflow Engine
15. Explainability
16. Observability
17. Security
18. Testing Strategy
19. Deployment
20. Extending the Platform
21. Performance Considerations
22. Architectural Trade-offs
23. Lessons Learned
24. Future Evolution
25. Glossary
26. References
27. Diagram Registry

---

1. Executive Summary

1.1 What Is the Enterprise AI Platform?

The Enterprise AI Platform is a **reusable, modular monolith** implemented in Java 21 and Spring Boot 3.3. It provides the infrastructure for building AI-powered enterprise applications: retrieval-augmented generation (RAG), semantic enrichment, knowledge graph construction, multi-provider AI orchestration, evaluation, explainability, and production observability.

The platform is **not** a finished application. It is a **foundation** — a set of reusable modules, interfaces, and architectural patterns that support building AI-powered applications on top of a common core.

1.2 Why Does It Exist?

Most AI demonstrations are single-file Python scripts that call an LLM API. They lack modularity, testability, observability, and production resilience. The Enterprise AI Platform demonstrates how an experienced engineering team would integrate AI into enterprise software: with clear boundaries, abstract interfaces, graceful degradation, and full auditability.

The platform serves three purposes:

1. **Reference implementation**: Demonstrates production-grade AI engineering in Java
2. **Reusable foundation**: Supports multiple AI applications from a single codebase
3. **Portfolio demonstration**: Shows Staff Engineer-level architecture and engineering judgement

1.3 Scope

In scope:

- Multi-provider AI orchestration (Ollama, OpenAI-compatible)
- Retrieval-augmented generation (RAG) with hybrid search
- Semantic enrichment with automatic entity/concept extraction
- GraphRAG with Neo4j knowledge graph traversal
- Prompt registry with versioning and categories
- Model capability registry with intelligent routing
- Evaluation framework (grounding, faithfulness, hallucination detection)
- Full explainability metadata on every inference
- Workflow engine for multi-step document processing
- Production observability (Micrometer, Prometheus, health indicators)
- Graceful degradation for all external dependencies
- 157 automated tests across 5 testing layers

Out of scope:

- Microservices deployment (the platform is a modular monolith)
- Multi-agent AI systems
- Model fine-tuning infrastructure
- User management / multi-tenancy
- Billing / usage tracking
- Real-time collaboration

1.4 Target Use Cases

The platform supports building applications for:

- **Document Intelligence**: Upload, index, search, and analyze document collections
- **Contract Intelligence**: Extract obligations, parties, dates, and terms
- **Enterprise Search**: Hybrid search across organizational knowledge bases
- **Compliance**: Policy analysis, regulatory mapping, requirement extraction
- **Financial Analysis**: Report analysis, entity extraction, relationship mapping
- **Research Platforms**: Literature analysis, citation networks, concept discovery

The included **Document Intelligence** application serves as the first reference implementation. Future applications are built on the same platform core.

1.5 Non-Goals

The platform does **not** aim to be:

- A no-code AI platform (it is a developer framework)
- A SaaS product (it is self-hosted infrastructure)
- A chatbot framework (it is an enterprise AI platform)
- A model training/inference service (it orchestrates external providers)

Related ADRs: ADR-001], ADR-020]

---

2. Platform Vision

2.1 AI as Infrastructure

The central thesis of the platform is: **AI is infrastructure, not a feature**.

LLMs, embedding models, vector databases, and knowledge graphs are infrastructure components — analogous to databases, message queues, and caches. They should be abstracted behind interfaces, selected via configuration, and swapped without changing business logic.

This contrasts with the common pattern of embedding provider-specific code directly in application logic. In this platform, a `DocumentService` never calls `Ollama`. It calls `EnrichmentService`, which may be backed by Ollama, OpenAI, or a regex fallback.

2.2 Domain Independence

The platform core contains **no domain-specific logic**. There are no assumptions about legal documents, financial reports, or any specific industry. Domain customization happens through the `DomainConfiguration` SPI — a pluggable interface that provides concept definitions, analysis objectives, finding hierarchies, and system instructions.

The Document Intelligence reference application provides default configurations. New domains add their own `DomainConfiguration` implementations without modifying platform code.

Related ADR: ADR-017]

2.3 Platform vs Application

A critical architectural distinction runs through the entire codebase:

- **Platform modules** (`platform-audit` , `platform-auth` , `platform-document` , `platform-search` , `platform-ai` , `platform-neo4j` , `platform-workspace` , `platform-observability`) provide infrastructure. They define SPIs, orchestration, and reusable services.
- **Application module** (`platform-api`) wires the platform together into a runnable application. It provides REST controllers, Thymeleaf templates, and application-specific configuration.

Future applications (Contract Intelligence, Financial Analysis, Compliance) would create their own application modules that depend on the same platform modules.

2.4 Reference Application Concept

The Document Intelligence application demonstrates how to use the platform. It includes:

- Document upload and ingestion (PDF, DOCX, TXT, HTML)
- Semantic enrichment during ingestion
- Hybrid search with keyword, vector, and graph retrieval
- AI-powered question answering with full grounding and evaluation
- Workspace management with 5-phase workflow wizard
- Multi-language UI (English, German, French)
- Audit logging with correlation IDs
- Health endpoints and Prometheus metrics

This application is both functional and educational — it shows how to wire the platform together.

Related ADR: ADR-020]

---

3. Design Philosophy

3.1 Eight Governing Principles

The platform is governed by eight principles. Every architectural decision is evaluated against these. When a decision conflicts with a principle, it is either rejected or the principle is amended with explicit rationale.

#	Principle	ADR
1	**AI is Infrastructure** — LLMs, embeddings, and vector stores are abstracted behind interfaces	ADR-003
2	**Modular Monolith** — Compile-time module boundaries enforce dependency direction	ADR-001
3	**Graceful Degradation** — Every external dependency is optional; the platform starts with zero infrastructure	ADR-016
4	**Explainability by Default** — Every inference produces auditable metadata	ADR-012
5	**Documents Are the Knowledge Source** — Knowledge is extracted from documents during ingestion, never manually curated	ADR-007
6	**Domain Independence** — Platform core contains no domain-specific logic	ADR-017
7	**Testing as Architecture Documentation** — Tests validate behavior; test structure mirrors architecture	ADR-019
8	**Production Readiness** — Observability, health checks, and configuration are first-class concerns	ADR-015

3.2 Modularity

Module boundaries are enforced at **compile time** through Maven's dependency resolution. The dependency graph is directional — higher-level modules depend on lower-level APIs, never the reverse. `platform-api` is the assembly module that wires everything together; no other module depends on it.

```
platform-api
  ├── platform-ai
  │     ├── platform-search
  │     │     └── platform-document
  │     │           └── platform-audit
  │     ├── platform-neo4j
  │     └── platform-audit
  ├── platform-auth ── platform-audit
  ├── platform-workspace ── platform-document, platform-audit
  └── platform-observability
```

Each module has a single, well-defined responsibility. When a module grows too large (e.g., `platform-ai`), its internal package structure provides further separation without adding Maven modules.

3.3 Extensibility

Extension points are defined as **Service Provider Interfaces (SPIs)**. The platform defines the contract; implementations provide the behavior. Key SPIs:

SPI	Location	Purpose
`ChatCompletionProvider`	`platform-ai/api/`	LLM backends
`EmbeddingProvider`	`platform-search/api/`	Embedding generation
`GraphSearchProvider`	`platform-search/api/`	Graph-based retrieval
`VectorSearchProvider`	`platform-search/api/`	Vector similarity search
`EnrichmentService`	`platform-ai/api/`	Semantic enrichment
`EvaluationService`	`platform-ai/api/`	Answer evaluation
`DomainConfiguration`	`platform-ai/api/`	Domain-specific rules
`WorkflowEngine`	`platform-ai/api/`	Process orchestration

New implementations are activated by `@Component` + `@ConditionalOnProperty`. No existing code changes are needed.

3.4 Configuration Over Specialization

Provider selection, model routing, prompt choice, and retrieval strategy are all driven by **configuration**, not code. The `ProviderRouter` selects providers based on model name prefix and capability registry data. The `RetrievalOrchestrator` selects strategies based on classified intent. Conditional beans activate and deactivate based on property presence.

This means the platform can serve different domains with different providers simply by changing `application.yml` — no code changes, no rebuilds.

3.5 Testing Philosophy

Tests are **executable architecture documentation**. A Staff Engineer should understand how the platform works by reading the test suite. Tests describe behavior, not implementation:

```
✅ "routes openai:gpt-4o to openai provider"
✅ "skips unavailable provider and selects available one"

❌ "testProviderRouter"
❌ "testGetLatest"
```

The 157 tests span 5 layers: unit, integration, architecture, contract, and browser (Playwright). The test structure mirrors the platform architecture.

Related ADRs: ADR-020], ADR-019]

---

4. System Context

The Enterprise AI Platform is a single deployable Spring Boot application communicating with four external systems: PostgreSQL (required), Qdrant (optional vector DB), Neo4j (optional graph DB), and Ollama/OpenAI (optional LLM). All optional dependencies degrade gracefully.

[Architecture Diagram 01 — System Context: Show platform as central box with external actors (Browser, Ollama, OpenAI, Qdrant, Neo4j, PostgreSQL) connected via labeled arrows indicating protocol (HTTP REST, Bolt, JDBC).]

Technology Stack

Layer	Technology	Version
Language	Java	21
Framework	Spring Boot	3.3.5
Database	PostgreSQL (prod), H2 (test)	pgvector:pg16
Vector DB	Qdrant	latest
Graph DB	Neo4j	5-community
LLM	Ollama (local), OpenAI-compatible (cloud)	—
Metrics	Micrometer + Prometheus	via Boot BOM
UI	Thymeleaf + Bootstrap 5	via Boot BOM
Testing	JUnit 5, Spring Boot Test, Playwright	—

Related: ADR-001, ADR-002

---

5. Module Overview

platform-audit (leaf): Immutable audit log. JDBC implementation. Correlation IDs. All other modules depend on it.

platform-auth: JWT (HS256) + BCrypt (strength 12) + refresh token rotation. Form login + stateless API auth.

platform-document: Document lifecycle. PDFBox/POI/JSoup text extraction. Scheduled ingestion worker (10s poll).

platform-search: Hybrid search with keyword (JPA/TF), vector (Qdrant), graph (Neo4j) fusion. Weighted linear combination (k*0.40 + v*0.40 + c*0.20). Sentence-aware chunking (1200-char target, 150-char overlap). Two-stage reranking.

platform-ai: Heart of the platform. 31 interfaces across RAG orchestration, enrichment, evaluation, prompt registry (8 prompts, 6 categories), model capability registry (6 models), provider router (4-tier selection), retrieval orchestrator, workflow engine, DomainConfiguration SPI.

platform-neo4j: Knowledge graph persistence. Auto-generated during ingestion. NodeProvenance on every element. Optional (conditional on platform.neo4j.uri).

platform-workspace: 5-phase wizard (SETUP->INGESTION->ANALYSIS->REVIEW->COMPLETE). Document linking, timeline events.

platform-observability: Micrometer + Prometheus. Health indicators (Ollama, Qdrant, providers). AiMetrics. Reusable across applications.

platform-api: Assembly module. REST controllers, Thymeleaf UI, SearchInfrastructureConfig for conditional bean wiring.

Related: ADR-001

---

6. Runtime Architecture

Startup: Component scan -> @ConfigurationProperties binding -> @ConditionalOnProperty evaluation -> Registry seeding -> Health indicator discovery -> Tomcat start.

DI: Constructor injection exclusively. Optional deps via ObjectProvider. No field injection.

Provider discovery: @Component beans gated by @ConditionalOnProperty. ProviderRouter receives List. Selection: model prefix -> capability -> preferred -> first available.

Related: ADR-002, ADR-003, ADR-016

---

7. Document Lifecycle

[Architecture Diagram 03 — Document Ingestion Pipeline: Sequence diagram showing Upload -> DocumentService -> IngestionWorker -> IngestionProcessor -> TextExtraction -> Enrichment -> Chunking -> Embedding -> PostgreSQL+Qdrant+Neo4j.]

Pipeline Steps

1. **Upload**: HTTP POST -> DocumentService.createDocument() creates Document + IngestionJob (PENDING)
2. **Scheduled Poll**: DocumentIngestionWorker polls every 10s for PENDING jobs
3. **Processing**: DefaultDocumentIngestionProcessor dispatches to IndexingOrchestrationService or keyword-only fallback
4. **Extraction**: TextExtractionService (PDFBox for PDF, POI for DOCX, JSoup for HTML)
5. **Enrichment**: EnrichmentHook extracts entities/concepts/relationships via EnrichmentService (LLM or regex), persists to Neo4j if available
6. **Chunking**: SentenceAwareChunkingStrategy splits at sentence boundaries (1200-char target, 150-char overlap)
7. **Embedding**: OllamaEmbeddingProvider generates 768-dim vectors via Ollama /api/embeddings
8. **Persistence**: Chunks to PostgreSQL (JPA), vectors to Qdrant (REST), graph to Neo4j (Bolt)

Graceful Degradation in Ingestion

- No embedding config -> keyword-only indexing (no vectors)
- No Qdrant -> vectors skipped, keyword search works
- No Neo4j -> enrichment runs, graph persistence skipped
- No LLM -> regex-based enrichment fallback

Related: ADR-007, ADR-010, ADR-016

---

8. AI Architecture

[Architecture Diagram 04 — AI Inference Pipeline: Sequence diagram showing User Query -> Intent Classification -> Retrieval Strategy -> Search (keyword+vector+graph) -> Fusion -> Reranking -> Prompt Assembly -> ProviderRouter -> LLM -> Grounding -> Validation -> Evaluation -> Explainability -> Structured Answer.]

Full AI Pipeline

1. **Intent Classification**: QueryIntentClassifier.classify()
2. **Strategy Selection**: RetrievalOrchestrator maps intent to SearchMode
3. **Retrieval**: RetrievalAugmentationService -> SearchFacade -> HybridRetrievalService (keyword+vector+graph fusion)
4. **Context Assembly**: ContextAssembler builds PromptContext
5. **Prompt Building**: PromptBuilder renders template via PromptRegistry
6. **Provider Selection**: ProviderRouter selects ChatCompletionProvider
7. **LLM Inference**: ChatCompletionProvider.complete() -> raw answer
8. **Validation**: TemporalConsistencyValidator + ClaimValidator
9. **Grounding**: GroundingService -> ConfidenceProfile
10. **Evaluation**: EvaluationService -> grounding, faithfulness, hallucination scores
11. **Explainability**: RetrievalOrchestrationResult captures full metadata
12. **Response**: AiResponse(ReasonedAnswer, InferenceMetadata)

Provider SPI

Interface	Implementations	Activation
ChatCompletionProvider	OllamaChatProvider, OpenAiChatProvider	@ConditionalOnProperty
EmbeddingProvider	OllamaEmbeddingProvider	@ConditionalOnProperty
RerankingProvider	OllamaRerankingProvider	@ConditionalOnProperty
VectorSearchProvider	QdrantVectorSearchProvider, NoOpVectorSearchProvider	Conditional on host
GraphSearchProvider	Neo4jGraphSearchAdapter, NoOpGraphSearchProvider	Conditional on Neo4j

Related: ADR-003, ADR-004, ADR-008, ADR-011

---

9. Semantic Enrichment

[Architecture Diagram 05 — Semantic Enrichment Engine: Data flow diagram showing Document Text -> Entity Extraction (ORGANIZATION, PERSON, TECHNOLOGY, REGULATION) -> Concept Extraction (temporal, financial, domain) -> Relationship Extraction (REFERENCES, PART_OF, RELATED_TO) -> Graph Persistence (Neo4j with NodeProvenance).]

The Semantic Enrichment Engine automatically extracts structured knowledge from documents during ingestion. Users never manually create knowledge entries. The document corpus IS the knowledge source.

Extraction Pipeline

1. **Entity Extraction**: Regex patterns for ORGANIZATION, PERSON, DATE, MONEY. LLM-based extraction via entity-extraction/v1 prompt when available.
2. **Concept Extraction**: Domain-specific concepts with confidence scores and related entities.
3. **Relationship Extraction**: Typed relationships (REFERENCES, PART_OF, RELATED_TO, MENTIONS) between entities and concepts.

EnrichmentHook

Bridges enrichment results to Neo4j: EnrichmentContext -> EnrichmentResult -> GraphNode/GraphRelationship -> Neo4j. Runs as part of DefaultDocumentIngestionProcessor. Gracefully degrades when enrichment or Neo4j is unavailable.

Provenance

Every node and relationship carries NodeProvenance: sourceDocumentId, chunkId, chunkOffset, extractionConfidence, extractionTimestamp, extractionModel, promptVersion, provider.

Related: ADR-007, ADR-018

---

10. GraphRAG

[Architecture Diagram 06 — GraphRAG Retrieval Flow: Three parallel retrieval streams (Keyword, Vector, Graph) converging into Fusion -> Reranking -> Candidates. Show graph stream as optional with dashed border. Show Neo4jGraphSearchAdapter bridging GraphEnrichmentService to GraphSearchProvider SPI.]

GraphRAG extends retrieval with knowledge graph traversal. When Neo4j is available, the retrieval orchestrator includes graph results alongside keyword and vector results in the fusion step.

Architecture

Neo4jGraphSearchAdapter bridges platform-neo4j (GraphEnrichmentService) to platform-search (GraphSearchProvider SPI). This adapter:

- Implements GraphSearchProvider interface
- Activated by @ConditionalOnBean(GraphEnrichmentService.class)
- Calls GraphEnrichmentService.traverse() for graph traversal
- Returns RetrievalCandidate objects for fusion

Fusion Integration

Three-way merge in DefaultHybridRetrievalService:

- Keyword results enter at full weight
- Vector results enter at full weight
- Graph results boost existing candidates via combineWithGraph() (+15% max)
- Graph-only candidates receive baseline scores

Graceful Degradation

When Neo4j is unavailable: GraphEnrichmentService bean not created -> Neo4jGraphSearchAdapter not created -> NoOpGraphSearchProvider active -> graph search returns empty -> keyword+vector continue normally.

Related: ADR-008, ADR-009, ADR-016

---

11. Retrieval Architecture

[Architecture Diagram 07 — Retrieval Orchestration: Decision tree showing Query -> IntentClassifier -> {INDEX_INSPECTION: Keyword, QUESTION_ANSWERING: Hybrid, DOCUMENT_LOOKUP: Semantic} -> Search Execution -> Fusion -> Reranking -> Candidates.]

Intent Classification

QueryIntentClassifier.classify() maps natural language queries to intents using keyword rules. The classified intent drives strategy selection in RetrievalOrchestrator.

Strategy Selection (deterministic)

Intent	Strategy	Mode
INDEX_INSPECTION, CORPUS_DISCOVERY	KEYWORD_ONLY	KEYWORD
QUESTION_ANSWERING, WORKSPACE_ANALYSIS	HYBRID	HYBRID
DOCUMENT_RESEARCH, DOCUMENT_LOOKUP	SEMANTIC	SEMANTIC

Fusion Algorithm

Weighted linear combination: `score = (keyword*0.40 + vector*0.40 + confidence*0.20) * docTypeWeight + graphBoost`. Clamped to [0, 1]. Graph boost adds up to 15% for candidates with related graph nodes.

Reranking (two-stage)

1. DefaultRerankingService: sort by rankingScore descending
2. OllamaRerankingProvider (optional): LLM cross-encoder scores top 15 candidates, blends 80% LLM + 20% original score

Related: ADR-008, ADR-011

---

12. Prompt Architecture

Prompt Registry

Versioned prompts stored in DefaultPromptRegistry with 10 categories: RETRIEVAL, SUMMARIZATION, EXTRACTION, CLASSIFICATION, EVALUATION, REASONING, WORKFLOW, GRAPH, SEARCH, SYSTEM.

Prompt Lifecycle

1. Registered at startup in DefaultPromptRegistry constructor
2. Resolved by RetrievalOrchestrator via getLatest("rag-answer")
3. Rendered with variable substitution: {{context}}, {{question}}
4. Qualified ID recorded in RetrievalOrchestrationResult for audit

Categories

Category	Example Prompt	Use Case
RETRIEVAL	rag-answer/v1	RAG-grounded QA
EXTRACTION	entity-extraction/v1	Entity and concept extraction
EVALUATION	rerank-evaluation/v1	Cross-encoder relevance scoring
SYSTEM	intent-classification/v1	Query intent routing
GRAPH	graph-relation-extraction/v1	Knowledge graph population
SUMMARIZATION	document-summary/v1	Document summarization

Related: ADR-005

---

13. Model Architecture

Model Capability Registry

Seeded with 6 models (4 Ollama, 2 OpenAI) covering all capability dimensions: streaming, vision, JSON mode, tool calling, embeddings, reasoning, structured output, context window, output tokens, latency estimates, cost estimates, preferred use cases.

Provider Router (4-tier selection)

1. Model prefix: "openai:gpt-4o" -> openai provider
2. Capability match: request STREAMING -> provider with streaming-capable model
3. Preferred provider: explicitly requested provider
4. First available: first provider reporting isAvailable() == true

Adding a Model

```
registry.register(new ModelCapability("claude-3.5-sonnet", "anthropic",
    true, true, true, true, false, false, true,
    200000, 8192, 800, 3.0, 0.7,
    List.of("general", "analysis", "code"),
    List.of("cloud", "frontier")));
```

Adding a Provider

1. Implement ChatCompletionProvider
2. Add @Component + @ConditionalOnProperty
3. Provider appears automatically in ProviderRouter via List injection

Related: ADR-004, ADR-006

---

14. Workflow Engine

In-memory workflow engine with configurable step transitions. Two pre-registered workflows: document-intelligence (SETUP->INGESTION->ANALYSIS->REVIEW->COMPLETE, 5 steps) and batch-ingestion (INGEST->ENRICH->COMPLETE, 3 steps).

API: start(definitionId, context), advance(instanceId), previous(instanceId), findInstance(instanceId), listActive().

Steps declare handler types (manual, automated, ai) as extension points. Not yet enforced at runtime. Transitions are linear; conditional branching deferred to future.

Related: ADR-014

---

15. Explainability

Every inference produces RetrievalOrchestrationResult with: intent, strategy, provider, model, prompt template ID/version, timing (start/end), keyword/vector/graph result counts, fusion method, reranking status, trace log (step-by-step decisions), evaluation scores.

explain() method produces human-readable summary: "Intent: QUESTION_ANSWERING | Strategy: HYBRID | Provider: ollama | Model: qwen2.5:14b | Prompt: rag-answer/v1 | Sources: 15 | Fusion: weighted-linear-fusion | Reranking: yes | Duration: 245ms".

Related: ADR-012

---

16. Observability

Metrics (AiMetrics)

Metric	Type	Tags
ai.inference.duration	Timer (percentile histogram)	provider, model
ai.embedding.duration	Timer	provider, model
ai.retrieval.duration	Timer	mode
ai.graph.retrieval.duration	Timer	—
ai.enrichment.duration	Timer	—
ai.ingestion.duration	Timer	document_type
ai.provider.available	Gauge (0/1)	provider
ai.evaluation.*	Summary	metric name

Health Indicators

- OllamaHealthIndicator: GET / -> 200 = up
- QdrantHealthIndicator: GET /collections -> 200 = up

- ProviderHealthIndicator: aggregates ChatCompletionProvider.isAvailable() across all providers

Endpoints

- /actuator/health (with details)
- /actuator/prometheus
- /actuator/metrics
- /actuator/info

Related: ADR-015

---

17. Security

Authentication

JWT (HS256) via Nimbus JOSE + JWT library. BCrypt password hashing (strength 12). Refresh tokens SHA-256 hashed before storage. Token rotation on refresh. Configurable TTL: access 15min default, refresh 30d default.

Authorization

Role-based: USER, ANALYST, ADMIN. Stateless API auth (Authorization: Bearer header) + session-based form login (JSESSIONID). CSRF disabled (API-focused platform).

Provider Isolation

Provider API keys (OpenAI) stored in application.yml with env var substitution: ${OPENAI_API_KEY}. Never hardcoded. Ollama runs locally — no auth required.

Prompt Safety

Prompts are internal platform assets (not user-supplied). User input is injected as {{question}} variable into vetted prompt templates. HTML-escaping in controller responses prevents XSS.

Related: ADR-002

---

18. Testing Strategy

Summary of TESTING.md. 157 tests across 5 layers:

Layer	Count	Command	Purpose
Unit	44	mvn test	Isolated component behavior
Integration	91	mvn verify	Subsystem interaction with Spring context
Architecture	10	mvn verify	End-to-end behavioral validation
Contract	5	mvn verify	SPI stability protection
Playwright	12	mvn verify -Pui-tests	Browser user journeys

Test Corpus

test-corpus/ contains 3 documents (technical-spec, financial-report, contract-sample) with expected architectural behaviors. Tests validate enrichment, entity extraction, and concept detection against these documents.

CI Strategy

Default: mvn clean verify (unit + integration + architecture) UI profile: mvn verify -Pui-tests (Playwright, requires running app) Performance profile: mvn verify -Pperformance

Related: ADR-019, TESTING.md

---

19. Deployment

Requirements

- Java 21
- PostgreSQL (required, port 5433)
- Qdrant (optional, port 6333)
- Neo4j (optional, port 7687)
- Ollama (optional, port 11434)

Docker Compose

docker-compose.yml defines PostgreSQL + pgAdmin + Qdrant + Neo4j (graph profile). Start with: docker compose up -d (basic) or docker compose --profile graph up -d (with Neo4j).

Configuration

All configuration via application.yml with env var substitution: ${DB_URL:jdbc:postgresql://localhost:5433/enterprise_ai_platform}. Platform prefix for custom config: platform.auth.*, platform.ai.*, platform.search.*, platform.neo4j.*.

Startup

mvn spring-boot:run -pl platform-api (requires docker compose up -d first)

---

20. Extending the Platform

Adding an LLM Provider

1. Implement ChatCompletionProvider (providerName, isAvailable, complete)
2. Add @Component + @ConditionalOnProperty
3. Configure in application.yml under platform.ai.{provider}.*
4. Register model capabilities in DefaultModelCapabilityRegistry
5. ProviderRouter automatically discovers via List injection
6. Add health indicator by implementing HealthIndicator

Adding a Prompt

```
promptRegistry.register(new PromptTemplate("my-prompt", 1,
    PromptTemplate.Category.RETRIEVAL, "My prompt description",
    "Template with {{variable}}", List.of("variable"),
    "text", List.of("*"), 0.3, List.of(), Map.of()));
```

Adding a Workflow

```
definitions.put("my-workflow", new WorkflowDefinition(
    "my-workflow", "My Workflow", "Description",
    List.of(
        new WorkflowStep("start", "Start", "Begin", "manual", Map.of(), List.of("next")),
        new WorkflowStep("next", "Next", "Continue", "automated", Map.of(), List.of())
    ), "start"));
```

Adding a Domain

Implement DomainConfiguration interface. Provide as @Component or @Bean. Domain-specific concept definitions, objectives, finding hierarchies, centrality weights, and system instructions are automatically available to the AI pipeline.

Adding a Retrieval Source

Implement GraphSearchProvider (or similar SPI). Add @Component. DefaultHybridRetrievalService discovers via constructor injection. Results participate in fusion automatically.

Related: ADR-003, ADR-005, ADR-014, ADR-017

---

21. Performance Considerations

Chunking

- 1200-char target with sentence-boundary awareness
- 150-char overlap ensures context continuity
- Configurable via ChunkingStrategy SPI

Embedding

- Sequential embedding in current implementation
- Batch embedding API exists (embedBatch) but loops sequentially
- 768-dimensional vectors from nomic-embed-text
- Parallel embedding via virtual threads would improve throughput

Retrieval

- Weighted fusion is O(n) where n = total candidates
- Graph traversal depth limited to 2 hops
- No caching layer — each retrieval re-executes

LLM Inference

- Timeout configurable via platform.ai.ollama.request-timeout (default 120s)
- No streaming (returns complete response)
- No token-level latency tracking

Ingestion

- Scheduled worker polls every 10s, processes top 10 PENDING jobs
- No bulk/batch ingestion optimization
- Documents processed sequentially

---

22. Architectural Trade-offs

Modular Monolith vs Microservices

Decision: Modular Monolith. **Rationale**: Compile-time boundaries sufficient for current scale. No distributed systems complexity. Future extraction possible. (ADR-001)

Neo4j vs RDF Triple Store

Decision: Neo4j labeled property graph. **Rationale**: Cypher is more intuitive than SPARQL for traversal queries. No ontology management overhead. (ADR-009)

Qdrant vs pgvector

Decision: Qdrant as primary, pgvector as secondary. **Rationale**: Qdrant is purpose-built for vector search. pgvector works but Qdrant provides better performance at scale and native quantization. (ADR-010)

Provider Router vs Hardcoded Providers

Decision: Provider Router with SPI. **Rationale**: Adding providers requires zero changes to orchestration code. Selection is deterministic and auditable. (ADR-004)

Prompt Registry vs Inline Prompts

Decision: Registry with versioning. **Rationale**: Prompts are platform assets that evolve. Versioning enables audit trails and regression testing. (ADR-005)

Spring Boot vs Quarkus

Decision: Spring Boot 3.3. **Rationale**: Larger ecosystem, more familiar to enterprise teams, richer integration options. Quarkus offers faster startup but smaller community. (ADR-002)

Playwright vs Selenium

Decision: Playwright for browser tests. **Rationale**: Modern API, better reliability, auto-waits, parallel execution. Selenium is the legacy standard but Playwright provides better DX.

Weighted Linear Fusion vs Reciprocal Rank Fusion

Decision: Weighted linear fusion. **Rationale**: Simpler to implement and explain. RRF requires rank position tracking which is not available from all providers. The fusion method is accurately named in documentation. (ADR-008)

---

23. Lessons Learned

From Domain-Specific to Domain-Independent

The platform began as a German tenancy law analysis tool. Phase 1 removed domain-specific assumptions: 55 BGB paragraph numbers, 25+ German keywords, legal-specific enums and classifications. The result is a reusable platform. Lesson: domain-specific code is easier to remove when interfaces are clean.

Spring AI Removal

Spring AI 1.0.0 was added as a dependency but never used. Zero imports from org.springframework.ai.*. The build audit removed it and 5 transitive JARs. Lesson: regularly audit dependencies — unused libraries accumulate silently.

AI Pipeline Was Dead Code

The full AI pipeline (AiService with 12 collaborating services) compiled and passed all tests — but was never injected. The AiPageController only showed retrieval results without generating LLM answers. Lesson: bean wiring tests are critical for Spring applications.

GraphRAG Evolution

GraphRAG started as a documentation concept ("Neo4j participates in retrieval") but was never implemented. Phase 3 bridged the gap with Neo4jGraphSearchAdapter and wired it into the fusion pipeline. Lesson: document what IS implemented, not what SHOULD be implemented.

Prompt Registry Value

Prompt versioning was initially deferred but proved essential during testing — it enabled deterministic behavior validation and made prompt changes auditable. Lesson: version your prompts from day one.

Testing as Documentation

The test suite evolved from 128 wiring-verification tests to 157 behavioral tests that serve as executable architecture documentation. A Staff Engineer reading the tests can understand the platform without reading the implementation. Lesson: invest in test naming and structure.

---

24. Future Evolution

Implemented

- Modular monolith with 9 modules
- Multi-provider AI (Ollama, OpenAI-compatible)
- Prompt registry (8 prompts, 6 categories)
- Model capability registry (6 models)
- Semantic enrichment (entities, concepts, relationships)
- GraphRAG with Neo4j traversal
- Retrieval orchestration (intent-driven strategy)
- Evaluation (grounding, faithfulness, hallucination)
- Explainability metadata on every inference
- Workflow engine (2 pre-registered workflows)
- Production observability (Micrometer, health checks)
- Graceful degradation for all external dependencies
- DomainConfiguration SPI
- 157 automated tests across 5 layers

Partially Implemented

- DomainConfiguration: SPI interface exists but has no default implementation; services use hardcoded domain rules
- AiMetrics: Component exists at `platform-observability/ … /metrics/AiMetrics.java` but is not yet injected into any service
- WorkflowEngine: Engine exists (DefaultWorkflowEngine, in-memory) but no PhaseHandler implementations registered
- PhaseHandler: Interface exists in platform-workspace but no concrete implementations
- GraphRAG: Works but uses simple entity-name matching from queries (word-splitting), not graph embeddings or NER
- Playwright: 12 browser tests written but require running application (mvn verify -Pui-tests)

Future Ideas

- Dynamic model capability discovery from provider APIs
- Resilience4j integration (retry, circuit breaker)

- OpenTelemetry distributed tracing
- Multi-domain routing (select DomainConfiguration per query)
- Graph embedding models for semantic graph traversal
- Streaming LLM responses (SSE)
- Prompt A/B testing framework
- Automated evaluation regression testing
- Spring Modulith for runtime module verification
- GraalVM native image for reduced startup time

---

25. Glossary

Term	Definition
Capability	A feature an AI model supports (streaming, vision, JSON mode, embeddings)
Chunk	A segment of document text (~1200 chars) stored and indexed for retrieval
DomainConfiguration	SPI for providing domain-specific rules (concepts, objectives, instructions)
Embedding	A 768-dimensional vector representation of text generated by an embedding model
Enrichment	Automatic extraction of entities, concepts, and relationships from documents
Evaluation	Automated quality assessment of AI-generated answers
Explainability	Metadata describing how an AI inference was produced (provider, model, prompt, strategy)
Fusion	Combining retrieval results from multiple sources into a single ranked list
GraphRAG	Retrieval-augmented generation enhanced with knowledge graph traversal
Intent	The classified purpose of a user query (question answering, document lookup, etc.)
Modular Monolith	Single deployable with compile-time module boundaries
Prompt Registry	Versioned store of prompt templates with categories and metadata
Provenance	Metadata linking a graph element to its source document and extraction method
Provider	An AI backend (Ollama, OpenAI) that implements a platform SPI
Provider Router	Component that selects which provider to use for a given inference request
RAG	Retrieval-Augmented Generation — grounding LLM responses in retrieved documents
Registry	A store of known entities (prompts, models, capabilities) with query capabilities
Retrieval Strategy	The selection of which retrievers to use (keyword, vector, graph) for a query
SPI	Service Provider Interface — a pluggable contract for extending the platform
Workflow	A configurable multi-step process with defined state transitions

---

26. References

Internal

- Architecture Decision Records — 20 ADRs documenting every major decision
- TESTING.md — Testing philosophy, layers, and execution guide
- README.md — Project overview and quickstart

External Technologies

- Spring Boot 3.3 Reference
- Qdrant Documentation
- Neo4j Cypher Manual
- Ollama API
- OpenAI API Reference
- Micrometer Documentation
- Playwright Documentation

---

27. Diagram Registry

The following diagrams are maintained in `docs/diagrams/`. Completed diagrams are available as SVG; remaining diagrams are planned for the next documentation iteration.

ID	Title	Type	File	Status
01	System Context	C4 Level 1	`01-system-context.svg`	Complete
02	Container Architecture	C4 Level 2	`02-container-architecture.svg`	Complete
03	Module Dependencies	Component	`03-module-dependencies.svg`	Complete
04	AI Inference Pipeline	Sequence	`04-ai-inference-pipeline.svg`	Complete
05	Document Ingestion Pipeline	Sequence	—	Planned
06	Semantic Enrichment Engine	Data Flow	—	Planned
07	GraphRAG Retrieval Flow	Data Flow	`07-graphrag-retrieval.svg`	Complete
08	Provider SPI Architecture	Class Diagram	`17-provider-spi.svg`	Complete
09	Prompt Registry Structure	Entity Diagram	—	Planned
10	Model Capability Registry	Entity Diagram	—	Planned
11	Testing Layers	Pyramid	—	Planned
12	Deployment Architecture	Deployment	—	Planned

See `docs/diagrams/README.md` for diagram descriptions and conventions.

Appendix A — Architecture Decision Records

The following Architecture Decision Records document every major engineering decision made during the platform's development. Each ADR follows the standard format: Status, Context, Decision, Alternatives Considered, Consequences, Trade-offs, Future Evolution.

ADR-001-modular-monolith

Status

Accepted. Implemented in the Maven module structure at `pom.xml`.

Context

The Enterprise AI Platform must support multiple AI-powered applications (document intelligence, contract analysis, financial review, compliance, etc.) while remaining deployable as a single process. The platform must demonstrate clean separation of concerns without the operational complexity of microservices.

Decision

Adopt a **Modular Monolith** architecture with 9 Maven modules:

```
platform-audit       — Immutable audit log, correlation IDs
platform-auth        — JWT authentication, refresh token rotation
platform-document    — Document lifecycle, ingestion pipeline
platform-search      — Hybrid search (keyword + vector + graph)
platform-ai          — RAG orchestration, enrichment, evaluation, registry
platform-neo4j       — Auto-generated knowledge graph with provenance
platform-workspace   — Multi-phase workflow wizard
platform-observability — Micrometer metrics, health indicators
platform-api         — REST + Thymeleaf controllers, assembly
```

Module boundaries follow **dependency inversion**: higher-level modules depend on lower-level APIs, never the reverse. `platform-api` is the assembly module that wires everything together.

Alternatives Considered

- **Microservices**: Rejected. The platform demonstrates architecture without distributed systems complexity. Microservices add network boundaries, eventual consistency challenges, and deployment overhead that don't benefit a reference implementation.
- **Single JAR without modules**: Rejected. A flat structure would not enforce dependency direction or demonstrate separation of concerns.
- **OSGi modules**: Rejected. Adds runtime modularity complexity. Maven's compile-time enforcement is sufficient.

Consequences

- **Clear dependency direction**: `api → ai → search → document → audit` (audit is a leaf dependency)
- **Compile-time enforcement**: Modules cannot accidentally depend on each other
- **Single deployable**: One `platform-api` Spring Boot application with all modules on the classpath
- **Test isolation**: Each module can be tested independently

Trade-offs

- Module boundaries are enforced only at compile time, not runtime
- Adding a new module requires POM maintenance
- Some modules (audit) are cross-cutting dependencies of many others

Future Evolution

- Modules may split if responsibilities grow too large (e.g., `platform-ai` could become `platform-ai-core` + `platform-ai-providers`)
- Spring Modulith could be introduced for runtime module verification
- The architecture deliberately supports future extraction of modules into microservices if needed

See also: [[ADR-002]], [[ADR-019]], [[ADR-020]]

ADR-002-spring-boot

Status

Accepted. Implemented in `pom.xml` at `spring-boot-starter-parent:3.3.5`.

Context

The platform must be recognizable to experienced Java engineers, leverage mature ecosystems for security, persistence, and observability, and remain maintainable by teams familiar with enterprise Java.

Decision

Use **Spring Boot 3.3** with Java 21 as the application framework. All modules depend on `spring-boot-starter-parent` BOM for version management. Key Spring integrations:

- `spring-boot-starter-web` — REST controllers
- `spring-boot-starter-security` — JWT authentication via Nimbus JOSE
- `spring-boot-starter-data-jpa` — PostgreSQL/H2 persistence
- `spring-boot-starter-actuator` — Health endpoints, metrics

- `spring-boot-starter-thymeleaf` — Server-rendered UI pages
- `spring-boot-starter-validation` — Request validation
- `@ConfigurationProperties` — Type-safe configuration binding
- `@ConditionalOnProperty` / `@ConditionalOnBean` — Provider activation

Alternatives Considered

- **Quarkus**: Rejected. Smaller ecosystem, less familiar to enterprise teams, fewer library integrations.
- **Micronaut**: Rejected. Strong compile-time DI but smaller community and fewer reference architectures.
- **Plain Java with manual DI**: Rejected. Would require reimplementing what Spring provides (security, JPA, scheduling, Actuator, property binding).
- **Spring Boot 2.x**: Rejected. Java 21 virtual threads require Spring Boot 3.x.

Consequences

- **Rich ecosystem**: Spring Security, Spring Data JPA, Spring Actuator all integrated out of the box
- **Virtual threads**: Java 21 + Spring Boot 3.3 enable `spring.threads.virtual.enabled`
- **Conditional beans**: `@ConditionalOnProperty` enables graceful degradation for optional infrastructure
- **Configuration**: `@ConfigurationProperties` with `platform.*` prefix ensures consistent, type-safe config

Trade-offs

- Startup time is slower than compile-time DI frameworks
- Spring's "magic" can obscure wiring for newcomers (mitigated by explicit `@Bean` definitions in `SearchInfrastructureConfig`)
- Memory footprint is larger than minimalist frameworks

Future Evolution

- Spring Modulith could add runtime module verification
- Spring AI could replace custom provider HTTP clients when mature
- GraalVM native image compilation could reduce startup time for serverless deployments

See also: [[ADR-001]], [[ADR-003]], [[ADR-015]]

ADR-003-provider-abstraction

Status

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/api/ChatCompletionProvider.java` and related interfaces.

Context

AI backends (Ollama, OpenAI, Anthropic, Gemini) expose different APIs, authentication schemes, and request/response formats. Business logic must not depend on any specific provider implementation. Adding a new provider should require zero changes to orchestration code.

Decision

Define **SPI interfaces** for every AI capability. Implementations are conditionally activated based on configuration:

Interface	Implementations	Activation
`ChatCompletionProvider`	`OllamaChatProvider`, `OpenAiChatProvider`	`@ConditionalOnProperty`
`EmbeddingProvider`	`OllamaEmbeddingProvider`	`@ConditionalOnProperty`
`RerankingProvider`	`OllamaRerankingProvider`	`@ConditionalOnProperty`
`VectorSearchProvider`	`QdrantVectorSearchProvider`, `NoOpVectorSearchProvider`	Conditional on host
`GraphSearchProvider`	`Neo4jGraphSearchAdapter`, `NoOpGraphSearchProvider`	Conditional on Neo4j
`EvaluationService`	`DefaultEvaluationService`	Always active

The `ProviderRouter` (`DefaultProviderRouter`) selects a provider using a 4-tier strategy:

1. Model name prefix (`openai:gpt-4o` → openai)
2. Capability match (streaming → capable provider)
3. Preferred provider
4. First available fallback

Business services depend on interfaces, never on concrete implementations.

Alternatives Considered

- **Hardcoded provider selection in business logic**: Rejected. Would couple orchestration to specific providers and make adding new providers expensive.
- **Spring AI abstraction**: Rejected. At implementation time, Spring AI 1.0.0 provided limited control over provider selection and lacked the capability registry concept. The dependency was removed during build audit.
- **Factory pattern without SPI**: Rejected. An SPI with conditional beans is more idiomatic in Spring and supports auto-discovery.

Consequences

- **Provider-agnostic business logic**: `AiService` depends on `ChatCompletionProvider`, not Ollama or OpenAI
- **Conditional activation**: Providers activate only when their configuration is present
- **No-code provider addition**: Implement `ChatCompletionProvider` + `@Component` + `@ConditionalOnProperty`

- **Graceful degradation**: When no provider is available, `ProviderRouter` throws a clear exception

Trade-offs

- Each provider implements its own HTTP client (no shared HTTP infrastructure)
- Provider implementations must handle their own error translation
- Capability discovery is static (seeded in `DefaultModelCapabilityRegistry`) rather than dynamic

Future Evolution

- Dynamic capability discovery via provider API introspection
- Shared HTTP client infrastructure with configurable retry/timeout
- Provider cost and latency metadata for intelligent routing

See also: [[ADR-004]], [[ADR-006]], [[ADR-016]]

ADR-004-provider-router

Status

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/application/DefaultProviderRouter.java`.

Context

With multiple AI providers available (Ollama local, OpenAI cloud), the platform must select the appropriate provider for each inference request. Selection criteria include model name, required capabilities, user preference, and provider availability.

Decision

Implement a **Provider Router** (`ProviderRouter` interface) with a deterministic 4-tier selection strategy:

1. **Model prefix routing**: `openai:gpt-4o` routes to the OpenAI provider, `ollama:qwen2.5` routes to Ollama
2. **Capability-based selection**: If a specific capability is requested (e.g., `STREAMING`), the router selects a provider whose models support that capability, as defined in the `ModelCapabilityRegistry`
3. **Preferred provider**: If the caller specifies a preferred provider, it is selected if available
4. **First available fallback**: The first provider reporting `isAvailable() == true` is selected

If no provider is available, the router throws `IllegalStateException` with a clear message.

Business services call `router.routeChat(InferenceRequest)` — they request capabilities, not specific providers.

Alternatives Considered

- **Direct injection of a single provider**: Rejected. Would make multi-provider setups impossible and require code changes to switch providers.
- **Random/round-robin selection**: Rejected. Non-deterministic behavior is harder to debug and audit.
- **Cost/latency-aware routing**: Deferred to future evolution. The router architecture supports adding routing dimensions.

Consequences

- **Deterministic routing**: Same input always produces same routing decision
- **Auditable**: Every routing decision is logged (`log.debug("Routed '{}' to provider '{}' by model prefix", ...)`)
- **Extensible**: Adding a routing dimension means adding a strategy tier, not rewriting the router
- **Clear failure mode**: Unavailable providers are skipped with explicit fallthrough

Trade-offs

- Static capability registry requires manual updates when new models are added
- No runtime performance tracking for latency/cost-based routing
- Model prefix convention (`provider:model`) is a convention, not enforced by types

Future Evolution

- Cost-aware routing using `ModelCapability.estimatedCostPer1kTokens()`
- Latency-aware routing using `ModelCapability.estimatedLatencyMs()`
- Region-aware routing for multi-region deployments
- Dynamic capability discovery from provider APIs

See also: [[ADR-003]], [[ADR-005]], [[ADR-006]]

ADR-005-prompt-registry

Status

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/application/DefaultPromptRegistry.java`.

Context

AI prompts are a critical platform asset. Without versioning, prompt changes cannot be tracked, audited, or rolled back. Without a registry, prompts are scattered across Java string constants, making them impossible to discover or manage.

Decision

Implement a **Prompt Registry** (`PromptRegistry` interface) that stores versioned prompt templates with metadata:

Feature	Implementation
Versioning	`PromptTemplate.id` + `PromptTemplate.version` → qualified ID `"rag-answer/v1"`
Categories	`PromptTemplate.Category` : RETRIEVAL, SUMMARIZATION, EXTRACTION, CLASSIFICATION, EVALUATION, REASONING, WORKFLOW, GRAPH, SEARCH, SYSTEM
Variables	`{{variable}}` template substitution via `render(Map<String, String>)`
Metadata	`expectedOutputType` , `supportedModels` , `recommendedTemperature` , `examples`
Discovery	`findByCategory(Category)` , `listPromptIds()` , `getLatest(String)`

The default registry (`DefaultPromptRegistry`) seeds 8 prompts across 6 categories. Prompts are registered programmatically; in production, they would be loaded from YAML/JSON resources.

Every inference records which prompt was used (`RetrievalOrchestrationResult.promptTemplateId()`), enabling full reproducibility and audit trails.

Alternatives Considered

- **Hardcoded string constants**: Rejected. No versioning, no discovery, no audit trail.
- **Database-backed prompt store**: Deferred. Adds infrastructure dependency. In-memory registry with resource loading is sufficient for a reference implementation.
- **External prompt management service**: Rejected. Over-engineered for a modular monolith.

Consequences

- **Reproducibility**: Every inference records `promptTemplateId` + `promptTemplateVersion`
- **Discoverability**: `findByCategory()` enables prompt inventory
- **Safe evolution**: New prompt versions can be registered without deleting old ones
- **Regression testing**: Prompts are identifiable assets that can be tested

Trade-offs

- In-memory storage means prompts reset on restart (acceptable for a reference implementation)
- No prompt validation at registration time (variables are not checked against template)
- Rendering uses simple string substitution, not a template engine (deliberate simplicity)

Future Evolution

- YAML/JSON resource loading for external prompt definitions
- Prompt validation: verify all declared variables are used and all template variables are declared

- Prompt A/B testing: serve different versions to different users
- Prompt migration tooling for bulk updates

See also: [[ADR-003]], [[ADR-004]], [[ADR-012]]

ADR-006-model-capability-registry

Status

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/application/DefaultModelCapabilityRegistry.java`.

Context

Different AI models have different capabilities (streaming, vision, JSON mode, tool calling, embeddings). Business logic should not hardcode assumptions like "model X supports JSON." Instead, capabilities should be queryable through a central registry, enabling the `ProviderRouter` to make intelligent selection decisions.

Decision

Implement a **Model Capability Registry** (`ModelCapabilityRegistry` interface) that describes every available model's capabilities:

Capability	Examples
`supportsStreaming`	qwen2.5:7b (true), nomic-embed-text (false)
`supportsVision`	gpt-4o (true)
`supportsJson` / `supportsToolCalling`	gpt-4o, llama3.2 (true)
`supportsEmbeddings`	nomic-embed-text (true)
`supportsStructuredOutput`	gpt-4o (true)
`maxContextWindow`	128K (gpt-4o), 32K (qwen2.5)
`maxOutputTokens`	16384 (gpt-4o)
`estimatedLatencyMs`	500 (qwen2.5 local), 1200 (gpt-4o cloud)
`estimatedCostPer1kTokens`	$5.00 (gpt-4o), $0.00 (local Ollama)
`preferredUseCases`	"rag", "analysis", "summarization"

The registry supports querying by:

- Exact model name: `get("gpt-4o")`
- Provider: `findByProvider("ollama")`
- Capability: `findByCapability(EMBEDDING)`

- Provider + capability: `findByProviderAndCapability("ollama", STREAMING)`

The `DefaultModelCapabilityRegistry` seeds with 6 known models (4 Ollama, 2 OpenAI). The `ProviderRouter` uses the registry for capability-based routing decisions.

Alternatives Considered

- **Hardcoded if/else chains**: Rejected. Unmaintainable with growing model lists.
- **Runtime model discovery via API**: Deferred. Provider APIs (Ollama `/api/tags`, OpenAI `/models`) could populate the registry dynamically, but add latency and failure modes.
- **No registry — trust the provider**: Rejected. The platform needs to know capabilities before calling a model (e.g., "does this model support JSON mode?").

Consequences

- **Capability-aware routing**: `ProviderRouter` selects models based on required capabilities
- **Static seed data**: Registry is populated at startup with known models; new models require code changes
- **Query interface**: Rich query API enables future use cases (e.g., "find the cheapest model that supports vision")

Trade-offs

- Static data can become stale as providers add new models
- Estimated costs and latencies are approximations, not real-time measurements
- No runtime validation that the model actually supports claimed capabilities

Future Evolution

- Dynamic discovery from provider `/models` endpoints
- Runtime capability verification (send test prompts to verify claims)
- User-provided model registrations via configuration
- Integration with provider cost APIs for real-time pricing

See also: [[ADR-003]], [[ADR-004]], [[ADR-005]]

ADR-007-semantic-enrichment

Status

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/application/DefaultEnrichmentService.java` and `platform-api/src/main/java/com/cognitera/platform/api/ingestion/EnrichmentHook.java`.

Context

Documents contain unstructured information. To enable intelligent retrieval, the platform must extract structured knowledge — entities, concepts, and relationships — automatically during ingestion. Users should never manually populate knowledge bases.

Decision

Implement an automatic **Semantic Enrichment Engine** that runs as part of the document ingestion pipeline:

```
Upload → Text Extraction → Enrichment → Chunking → Embedding → PostgreSQL + Qdrant
                                          ↓
                                    Neo4j Graph
```

The enrichment pipeline uses a dual-mode extraction strategy:

1. **LLM-based**: When a `ChatCompletionProvider` is available, prompts the LLM with `entity-extraction/v1` template for high-quality structured extraction
2. **Regex fallback**: When no LLM is available, uses regex patterns for known entity types (ORGANIZATION, PERSON, DATE, MONEY)

Results are bridged to Neo4j via `EnrichmentHook`, which converts `EnrichmentContext` to `GraphNode` / `GraphRelationship` objects and persists them through `GraphEnrichmentService`.

Entities carry **provenance**: `sourceDocumentId`, `chunkId`, `extractionConfidence`, `extractionTimestamp`, `extractionModel`, `promptVersion`, `provider`.

Alternatives Considered

- **Manual knowledge entry (old platform-knowledge module)**: Rejected. Users should not manually curate knowledge. The document corpus is the knowledge source. The old `platform-knowledge` module was removed in Phase 1.
- **LLM-only enrichment**: Rejected. Would fail when no LLM is available. Regex fallback ensures basic enrichment always works.
- **No enrichment**: Rejected. Without enrichment, retrieval is purely lexical/semantic with no structured understanding.

Consequences

- **Automatic knowledge extraction**: Entities, concepts, and relationships are extracted during ingestion without user intervention
- **Graceful degradation**: Regex fallback when LLM is unavailable
- **Provenance**: Every graph node links back to its source document and extraction method
- **Neo4j population**: The knowledge graph is auto-generated, never manually curated

Trade-offs

- Regex patterns are less accurate than LLM extraction
- The regex patterns are English-centric and need extension for multilingual documents
- Enrichment adds latency to the ingestion pipeline

Future Evolution

- Multilingual entity extraction patterns
- Domain-specific enrichment via `DomainConfiguration`
- Confidence threshold configuration for entity filtering
- Parallel enrichment for large documents
- Integration with external NER services

See also: [[ADR-008]], [[ADR-009]], [[ADR-017]], [[ADR-018]]

ADR-008-graphrag

Status

Accepted. Implemented in `Neo4jGraphSearchAdapter` bridging `GraphEnrichmentService` to `GraphSearchProvider`, with integration in `DefaultHybridRetrievalService`.

Context

Traditional RAG retrieves chunks via keyword and vector similarity. However, semantically related documents may not share keywords or embedding proximity. A knowledge graph connecting entities, concepts, and documents enables traversal-based discovery that complements keyword and vector retrieval.

Decision

Implement **GraphRAG** — graph-enhanced retrieval — as an optional third retrieval source alongside keyword and vector:

```
SearchQuery
  ├── KeywordSearchProvider.search()  → keywordResults
  ├── VectorSearchProvider.search()   → vectorResults
  ├── GraphSearchProvider.search()    → graphResults (optional)
  └── merge(keyword, vector, graph)   → fused candidates
       ├── combine(): weighted linear fusion (k×0.40 + v×0.40 + c×0.20)
       └── combineWithGraph(): graph boost (existing + graph×0.15)
  → Reranking → LLM
```

Graph results **boost** existing candidates rather than replacing them. The `combineWithGraph()` method adds up to 15% score increase when graph traversal finds related nodes.

Graph retrieval is **optional**: `SearchMode.GRAPH` and `SearchMode.HYBRID_GRAPH` activate it. When Neo4j is unavailable, `NoOpGraphSearchProvider` returns empty results and retrieval continues with keyword + vector.

Alternatives Considered

- **Graph-only retrieval**: Rejected. Graph traversal is useful for discovery but less precise for exact match queries.
- **Graph as primary source with keyword/vector as secondary**: Rejected. Documents without graph entities would be invisible.
- **No graph retrieval**: Rejected. The enrichment engine already populates Neo4j; not using it for retrieval wastes the enrichment investment.

Consequences

- **Three-source fusion**: Keyword, vector, and graph results are merged in a single pipeline
- **Graceful degradation**: Graph retrieval is optional; platform works without Neo4j
- **Graph boost**: Related entities and concepts boost document scores by up to 15%
- **Explainability**: Graph participation is tracked in retrieval metadata

Trade-offs

- Graph search uses simple entity name matching from the query (not embedded query → nearest neighbor in graph embedding space)
- The `Neo4jGraphSearchAdapter` creates synthetic `ChunkReference` objects for graph results, which have lower fidelity than real chunk references
- Graph boost of 15% is fixed, not calibrated per domain

Future Evolution

- Graph embedding models for semantic graph traversal
- Configurable graph boost factors per domain
- Multi-hop reasoning via graph traversal patterns
- Graph-native reranking using centrality metrics

See also: [[ADR-007]], [[ADR-009]], [[ADR-011]]

ADR-009-neo4j

Status

Accepted. Implemented in `platform-neo4j/src/main/java/com/cognitera/platform/neo4j/service/GraphEnrichmentService.java`.

Context

The platform needs to store structured knowledge extracted from documents — entities, concepts, and their relationships. This data is inherently graph-shaped: entities relate to documents, concepts relate to entities, documents cite other documents. A relational database would require complex recursive CTEs for graph traversal.

Decision

Use **Neo4j** as the persistence layer for the automatically-generated knowledge graph. Neo4j is **not** a general-purpose database in this architecture — it stores only semantic enrichment output:

Node Type	Example
`DOCUMENT`	Uploaded PDF, DOCX, TXT
`ENTITY`	Organization, Person, Technology
`CONCEPT`	Temporal concepts, financial amounts, topics

Relationship	Example
`MENTIONS`	Document → Entity
`RELATED_TO`	Document → Concept
`BELONGS_TO`	Entity → Concept
`REFERENCES`	Document → Document

The graph is **auto-generated during ingestion** — never manually curated. The old `platform-knowledge` module (manual CRUD knowledge base) was removed in Phase 1.

Neo4j is **optional**: the `graph` profile in `docker-compose.yml` and `@ConditionalOnProperty(name = "platform.neo4j.uri")` ensure the platform starts without it.

Alternatives Considered

- **PostgreSQL with recursive CTEs**: Rejected. Graph traversal queries become complex and slow beyond 2-3 hops. Cypher is purpose-built for graph traversal.
- **Property graph in application memory**: Rejected. Does not persist across restarts; cannot scale beyond small corpora.
- **RDF triple store**: Rejected. Adds complexity (SPARQL, ontology management) without clear benefit over labeled property graphs for this use case.
- **No graph database**: Rejected. The enrichment engine produces graph-shaped data; storing it relationally would violate the "right tool for the data shape" principle.

Consequences

- **Native graph traversal**: `GraphEnrichmentService.traverse(seedIds, maxDepth)` uses Cypher for multi-hop queries

- **Optional infrastructure**: Platform works without Neo4j
- **Provenance**: Every node carries `NodeProvenance` linking back to source document and extraction method
- **Auto-generated**: Graph population happens during ingestion, not through user interaction

Trade-offs

- Adds infrastructure dependency when GraphRAG is desired
- Neo4j Community Edition has single-database limitation
- Graph schema is implicit (defined by code) rather than explicit (defined by constraints)

Future Evolution

- Graph embedding generation in Neo4j (GDS library)
- Cypher query templates for common traversal patterns
- Graph-native reranking using PageRank or centrality algorithms
- Multi-tenancy via Neo4j database-per-tenant (requires Enterprise)

See also: [[ADR-007]], [[ADR-008]], [[ADR-018]]

ADR-010-qdrant

Status

Accepted. Implemented in `platform-search/src/main/java/com/cognitera/platform/search/application/qdrant/QdrantVectorSearchProvider.java`.

Context

Vector search requires a database optimized for high-dimensional similarity queries. General-purpose databases (PostgreSQL) can support vectors via extensions (pgvector) but are not optimized for vector-first workloads.

Decision

Use **Qdrant** as the dedicated vector database. Qdrant is purpose-built for vector similarity search with:

- Native cosine similarity scoring
- Payload filtering (metadata alongside vectors)
- Collection management with configurable vector dimensions
- REST + gRPC APIs

The `QdrantVectorSearchProvider` communicates via REST API (`/collections/{name}/points/search`). Each vector point carries a payload with `chunkId`, `documentId`, `title`, `documentType`, `category`, `tags`, and `source`.

Qdrant is **optional**: when `platform.search.qdrant.host` is not configured, `NoOpVectorSearchProvider` provides empty search results and keyword search continues.

Configuration is managed via `QdrantProperties` (`@ConfigurationProperties(prefix = "platform.search.qdrant")`). The default collection is `enterprise_ai_chunks` with 768-dimensional vectors (matching `nomic-embed-text` output).

Alternatives Considered

- **pgvector (PostgreSQL extension)**: Rejected as primary vector store. While pgvector works for small-to-medium corpora, Qdrant provides better performance at scale, native quantization, and is purpose-built for vector search. pgvector is used for development/testing via H2 compatibility.
- **Weaviate, Pinecone, Milvus**: Rejected. Qdrant was chosen for its Rust performance, simple deployment (single binary), and REST API. The `VectorSearchProvider` SPI makes switching straightforward.
- **In-memory vector search**: Rejected. Does not persist across restarts.

Consequences

- **High-performance vector search**: Cosine similarity with payload filtering
- **Collection auto-creation**: `QdrantCollectionManager.ensureCollectionExists()` on startup
- **Batch indexing**: `indexBatch()` for efficient bulk ingestion
- **Graceful degradation**: Platform works without Qdrant (keyword-only search)

Trade-offs

- Additional infrastructure dependency in production
- REST API adds ~5-10ms latency vs gRPC
- No built-in quantization or disk-based indexing in the current configuration

Future Evolution

- gRPC client for lower latency
- Qdrant quantization for memory efficiency with large corpora
- Multi-collection strategy per tenant or document type
- Hybrid search pushdown to Qdrant (if Qdrant adds text search)

See also: [[ADR-008]], [[ADR-011]]

ADR-011-retrieval-orchestration

Status

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/application/DefaultRetrievalOrchestrator.java`.

Context

Different query types benefit from different retrieval strategies. A factual lookup ("What was the revenue in Q2?") benefits from keyword search. An exploratory question ("What capabilities does the platform have?") benefits from hybrid search. An index inspection query should not waste resources on vector search.

Decision

Implement a **Retrieval Orchestrator** that selects the retrieval strategy based on classified query intent:

```
User Query → Intent Classification → Strategy Selection → Search Execution → Result
```

Strategy mapping (deterministic and explainable):

QueryIntent	SearchMode	Rationale
`INDEX_INSPECTION`, `CORPUS_DISCOVERY`	`KEYWORD`	Exact match queries; vector search unnecessary
`QUESTION_ANSWERING`, `WORKSPACE_ANALYSIS`, `SOURCE_ANALYSIS`	`HYBRID`	Complex queries benefit from keyword + vector fusion
`DOCUMENT_RESEARCH`, `DOCUMENT_LOOKUP`	`SEMANTIC`	Conceptual queries benefit from vector similarity

The orchestrator produces `RetrievalOrchestrationResult` with full **explainability metadata**: intent, strategy, mode, prompt template, result counts, fusion method, reranking status, timing, and a step-by-step `traceLog`.

Alternatives Considered

- **Always run all retrievers**: Rejected. Wastes resources on inappropriate strategies (e.g., vector search for exact ID lookups).
- **User-specified strategy**: Rejected. Users should not need to understand retrieval internals.
- **ML-based strategy selection**: Deferred. A trained classifier could outperform keyword-based intent classification, but keyword rules are deterministic, explainable, and sufficient for initial release.

Consequences

- **Deterministic**: Same query always produces same strategy
- **Explainable**: Every retrieval decision is recorded in `traceLog`
- **Efficient**: Only appropriate retrievers are executed
- **Prompt-aware**: Retrieves the latest prompt template from `PromptRegistry`

Trade-offs

- Keyword-based intent classification can misclassify queries
- Strategy mapping is static (no runtime adaptation based on result quality)
- Graph retrieval is not automatically selected by the orchestrator (requires explicit `HYBRID_GRAPH` mode)

Future Evolution

- ML-based intent classification for higher accuracy
- Feedback loop: if results are poor, retry with expanded strategy
- Graph strategy auto-selection when enriched entities are detected in the query

See also: [[ADR-005]], [[ADR-008]], [[ADR-012]]

ADR-012-explainability

Status

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/model/RetrievalOrchestrationResult.java` and `platform-ai/src/main/java/com/cognitera/platform/ai/model/InferenceMetadata.java`.

Context

AI systems must be auditable. When the platform generates an answer, stakeholders need to know: which model generated it, which prompt template was used, which documents were retrieved, which retrieval strategy was selected, and how long each step took. Without explainability, AI output is a black box.

Decision

Make **explainability a first-class concern** on every inference. Every AI response carries `InferenceMetadata` and every retrieval carries `RetrievalOrchestrationResult` with:

Metadata	Example
`provider`	"ollama"
`model`	"qwen2.5:14b"
`promptTemplateId`	"rag-answer/v1"
`retrievalStrategy`	"HYBRID"
`keywordResultCount`	12
`vectorResultCount`	8
`graphNodeCount`	3
`totalSourceCount`	15
`fusionMethod`	"weighted-linear-fusion"
`rerankingApplied`	true
`rerankingProvider`	"ollama-cross-encoder"
`retrievalStartedAt` / `retrievalCompletedAt`	timestamps
`traceLog`	["Orchestration started", "Intent: QUESTION_ANSWERING", ...]
`evaluationScores`	{grounding: 0.72, faithfulness: 0.85}

The `explain()` method produces a human-readable summary:

```
Intent: QUESTION_ANSWERING | Strategy: HYBRID | Prompt: rag-answer/v1 | Sources: 15 | Fusion: weigh
```

Explainability is **built into the orchestration layer**, not bolted on as an afterthought. Business services don't call explainability APIs — the orchestrator populates metadata automatically.

Alternatives Considered

- **Optional explainability**: Rejected. Explainability should never be optional in an enterprise AI platform.
- **Separate explainability service**: Rejected. Would require duplicating orchestration state. Building metadata into the orchestration result ensures consistency.
- **User-facing explainability only**: Rejected. Internal diagnostics are as important as user-facing explanations.

Consequences

- **Every inference is auditable**: Full traceability from query to answer
- **Deterministic**: Same query → same strategy → same explainability output
- **Low overhead**: Metadata is collected during normal execution, not as a separate pass
- **Future-proof**: New retrieval sources automatically appear in metadata

Trade-offs

- `traceLog` is append-only string list (not structured log entries)
- `RetrievalOrchestrationResult` uses a builder with 19 setters (verbose but explicit)
- Metadata is not yet exposed through a dedicated API endpoint

Future Evolution

- Structured trace events with typed metadata (not string list)
- Explainability REST API (`GET /api/inferences/{id}/explain`)
- Visualization of retrieval decisions (Sankey diagram of query → strategy → results)
- Differential explainability: "why was document A ranked above document B?"

See also: [[ADR-011]], [[ADR-013]], [[ADR-014]]

ADR-013-evaluation

Status

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/application/DefaultEvaluationService.java`.

Context

AI-generated answers must be evaluated for quality. Without evaluation, there is no feedback loop to detect hallucination, poor grounding, or irrelevant answers. Evaluation must run automatically as part of the inference pipeline, not as a separate manual process.

Decision

Implement an **Evaluation Engine** that automatically evaluates every retrieval + inference cycle:

Metric	Method	Range
`groundingScore`	Context length vs answer length ratio	[0, 1]
`citationCoverage`	Citation markers [1], [2] in answer	[0, 1]
`faithfulness`	Inverse of hallucination indicators	[0, 1]
`answerRelevance`	Term overlap between question and answer	[0, 1]
`contextRelevance`	Term overlap between question and context	[0, 1]
`hallucinationIndicators`	Uncertainty phrase detection	integer ≥ 0
`passed`	Composite quality gate	boolean

The `DefaultEvaluationService` uses heuristic methods (regex patterns, term overlap, length ratios). In production, a dedicated evaluation LLM or framework (deepeval, ragas) would replace these heuristics.

Evaluation is wired into `DefaultRetrievalOrchestrator` — it runs after retrieval and before the result is returned. Every `RetrievalOrchestrationResult` carries evaluation scores in its metadata.

Alternatives Considered

- **No evaluation**: Rejected. An AI platform without quality feedback is irresponsible engineering.
- **LLM-as-judge only**: Deferred. Requires a second LLM call, adding latency and cost. Heuristic evaluation provides fast, deterministic feedback. LLM evaluation can be added as a separate evaluation profile.
- **Human evaluation only**: Rejected. Does not scale and cannot run in CI pipelines.

Consequences

- **Automatic quality gate**: Every retrieval is scored; low-quality retrievals are flagged
- **Deterministic**: Heuristic methods produce consistent, reproducible scores
- **Lightweight**: Evaluation adds negligible latency (no LLM calls)

Trade-offs

- Heuristic methods are less accurate than LLM-based evaluation
- `hallucinationIndicators` uses regex patterns that miss sophisticated hallucinations
- No benchmark dataset integration for regression testing

Future Evolution

- LLM-as-judge evaluation (separate profile, gated by configuration)
- RAG evaluation framework integration (deepeval, ragas)
- Evaluation benchmark dataset with ground truth annotations
- Automated regression testing: "does this prompt change improve or degrade evaluation scores?"

See also: [[ADR-011]], [[ADR-012]], [[ADR-014]]

ADR-014-workflow-engine

Status

Accepted. Implemented in `platform-ai/src/main/java/com/cognitera/platform/ai/application/DefaultWorkflowEngine.java`.

Context

Enterprise AI applications involve multi-step processes: document upload → extraction → enrichment → analysis → review → completion. These processes have defined steps, transitions, and state. Hardcoding step logic in controllers couples workflow to HTTP concerns. A reusable workflow engine separates process definition from execution.

Decision

Implement a reusable **Workflow Engine** (`WorkflowEngine` interface):

Concept	Implementation
`WorkflowDefinition`	Named workflow with ordered steps and transitions
`WorkflowStep`	Step with id, name, description, handler type (manual/automated/ai)
`WorkflowInstance`	Running instance with current step, status, context

Two pre-registered workflows demonstrate the engine:

1. **`document-intelligence`**: SETUP → INGESTION → ANALYSIS → REVIEW → COMPLETE (5 steps)
2. **`batch-ingestion`**: INGEST → ENRICH → COMPLETE (3 steps)

The engine supports:

- `start(definitionId, context)` — creates a new instance at the initial step
- `advance(instanceId)` — moves to the next step; marks COMPLETED when no further steps
- `previous(instanceId)` — returns to the previous step
- `findInstance(instanceId)` — retrieves instance state
- `listActive()` — finds all active instances

The current implementation is in-memory (suitable for single-node). A database-backed store would be needed for multi-node deployments.

Alternatives Considered

- **Hardcoded wizard in controller**: Rejected. The old workspace wizard was a hardcoded switch statement in `WorkspacePageController`. A reusable engine separates process logic from presentation.
- **Camunda/Flowable BPMN engine**: Rejected. Over-engineered for the current use case. The lightweight engine demonstrates the concept without framework lock-in.
- **Spring State Machine**: Rejected. Adds framework dependency for a relatively simple state transition model.

Consequences

- **Reusable**: Any module can define and execute workflows
- **Simple API**: 5 methods, intuitive lifecycle
- **Pre-registered workflows**: Demonstrates real usage without configuration files

Trade-offs

- In-memory storage means workflows reset on restart
- No persistence or fault tolerance for long-running workflows
- No parallel step execution or conditional branching
- Step handler types are declarative (manual/automated/ai) but not enforced

Future Evolution

- Database-backed instance storage for production durability
- Conditional transitions based on step outcomes
- Timer-based transitions (escalation after timeout)
- Visual workflow definition (BPMN subset)
- Workflow event publishing for audit integration

See also: [[ADR-001]], [[ADR-020]]

ADR-015-ai-observability

Status

Accepted. Implemented in `platform-observability/src/main/java/com/cognitera/platform/observability/metrics/AiMetrics.java`.

Context

AI operations are complex, multi-step processes with multiple external dependencies. Without observability, diagnosing failures (is the LLM slow? is Qdrant down? is enrichment producing too few entities?) requires log diving. Production AI systems require metrics.

Decision

Implement **AI-specific observability** using Micrometer + Prometheus, integrated into a dedicated `platform-observability` module:

Metric	Type	Tags
`ai.inference.duration`	Timer (percentile histogram)	provider, model
`ai.embedding.duration`	Timer	provider, model
`ai.retrieval.duration`	Timer	mode (keyword/semantic/hybrid)
`ai.graph.retrieval.duration`	Timer	—
`ai.enrichment.duration`	Timer	—
`ai.ingestion.duration`	Timer	document_type
`ai.prompt.duration`	Timer	template
`ai.provider.available`	Gauge (0/1)	provider
`ai.embedding.count`	Counter	provider
`ai.enrichment.entities`	Counter	—
`ai.evaluation.*`	Summary	metric name

Metrics are exposed via `/actuator/prometheus` and `/actuator/health`. Health indicators check Ollama, Qdrant, and provider availability.

The `platform-observability` module is reusable across future applications. It depends only on Micrometer and Spring Boot Actuator — no platform-specific dependencies.

Alternatives Considered

- **Logging only**: Rejected. Logs are not aggregatable or queryable at scale. Metrics enable dashboards and alerting.
- **OpenTelemetry from day one**: Deferred. Micrometer provides a simpler API that can export to OTLP when OpenTelemetry is adopted.
- **Metrics in each module**: Rejected. Centralizing metrics in `platform-observability` ensures consistent naming, avoids duplication, and enables reuse.

Consequences

- **Operational visibility**: Every AI operation produces metrics

- **Reusable module**: Future applications get observability by depending on `platform-observability`
- **Standard format**: Prometheus exposition format enables Grafana dashboards
- **Health endpoints**: Liveness, readiness, and dependency health checks via Actuator

Trade-offs

- Metrics are defined but not yet fully wired into all services (`AiMetrics` is injected in fewer places than ideal)
- No custom Grafana dashboard definitions
- No alerting rules (would be added in deployment configuration, not in the platform)

Future Evolution

- Wire `AiMetrics` into all AI services (`AiService`, `SearchService`, `DefaultEnrichmentService`)
- OpenTelemetry exporter for distributed tracing
- Pre-built Grafana dashboard JSON
- Alert definitions for critical metrics (LLM latency > threshold, provider down)

See also: [[ADR-002]], [[ADR-016]]

ADR-016-graceful-degradation

Status

Accepted. Implemented throughout the platform via `@ConditionalOnProperty`, `@ConditionalOnBean`, `ObjectProvider`, and `NoOp*` fallback beans.

Context

The Enterprise AI Platform depends on multiple external services: Ollama (LLM), Qdrant (vector DB), Neo4j (graph DB), PostgreSQL (relational DB). In production, any of these may be unavailable. The platform must continue functioning with reduced capabilities rather than failing entirely.

Decision

Design every external dependency as **optional**. The platform uses multiple Spring mechanisms for graceful degradation:

Mechanism	Example	Behavior
`@ConditionalOnProperty`	`OllamaChatProvider` requires `platform.ai.ollama.base-url`	Bean not created if config absent
`@ConditionalOnBean`	`Neo4jGraphSearchAdapter` requires `GraphEnrichmentService`	Bean not created if Neo4j unavailable
`ObjectProvider<T>`	`DefaultDocumentIngestionProcessor` injects `ObjectProvider<EnrichmentHook>`	Null-safe getIfAvailable()
`NoOp*` fallbacks	`NoOpVectorSearchProvider`, `NoOpGraphSearchProvider`	Return empty results, never throw
`try-catch` wrappers	`Neo4jGraphSearchAdapter.search()`	Log warning, return empty list
`isAvailable()` checks	`GraphSearchProvider.isAvailable()`, `ChatCompletionProvider.isAvailable()`	Caller checks before invoking
`ProviderRouter` fallback	4-tier selection skips unavailable providers	Throws only when ALL unavailable

The degradation is **visible**: health indicators report status, metrics track failures, logs record the reason. The platform never silently degrades.

Alternatives Considered

- **Mandatory all services**: Rejected. Would prevent development without full infrastructure.
- **Circuit breaker only**: Rejected. Circuit breakers (Resilience4j) are valuable for transient failures but don't address the "service never configured" case. Conditional beans handle both.
- **Feature flags**: Rejected. Adds complexity. Conditional bean activation is simpler and more idiomatic in Spring.

Consequences

- **Platform starts with zero infrastructure** (except PostgreSQL for basic operation)
- **157 tests pass without any external services** (H2 in-memory, no Ollama/Qdrant/Neo4j)
- **Clear failure modes**: When a service is down, the reason is logged and surfaced in health endpoints
- **Gradual capability ramp**: Start with keyword search → add Qdrant for vector → add Neo4j for GraphRAG → add Ollama for LLM

Trade-offs

- Some operations silently return empty results (graph search) rather than surfacing errors to users
- No retry logic for transient failures (deferred to Resilience4j integration)
- Health indicators currently read env vars directly rather than Spring config (known drift, documented)

Future Evolution

- Resilience4j integration for retry, circuit breaker, rate limiter
- Health indicators refactored to inject `@ConfigurationProperties`
- Degradation event publishing to audit log
- User-facing capability indicators ("vector search unavailable")

See also: [[ADR-003]], [[ADR-008]], [[ADR-009]], [[ADR-010]], [[ADR-015]]

ADR-017-domain-configuration

Status

Accepted. Interface defined in `platform-ai/src/main/java/com/cognitera/platform/ai/api/DomainConfiguration.java`. Default implementations embedded in existing services.

Context

The platform must support multiple application domains (contract intelligence, financial analysis, regulatory compliance, technical documentation) from a single codebase. Domain-specific logic (concept definitions, analysis objectives, finding hierarchies, system instructions) must not be hardcoded in platform services. The platform core must remain domain-independent.

Decision

Define a **`DomainConfiguration` SPI** that encapsulates domain-specific AI behavior:

Method	Returns	Purpose
`domainId()` / `displayName()`	String	Identity
`concepts()`	`List<ConceptDefinition>`	Keywords + governing references per concept
`objectives()`	`List<ObjectiveDefinition>`	Analysis objectives with keywords
`findingRoleMapping()`	`Map<String, String>`	Reference → finding role mapping
`findingRelationships()`	`List<FindingRelationship>`	Relationships between findings
`centralityWeights()`	`Map<String, Double>`	Centrality weights for references
`peripheralReferences()`	`Set<String>`	References classified as peripheral
`roleKeywords()`	`Map<String, List<String>>`	Source role classification keywords
`systemInstruction()`	`String`	Domain-specific AI system instruction
`answerStructureGuidance()`	`String`	Domain-specific answer structure

Applications provide their `DomainConfiguration` as a Spring bean. The platform's existing services (`DefaultConceptExtractionService`, `DefaultObjectiveAnalysisService`, etc.) currently embed default configurations. These defaults represent the original document intelligence domain but should be migrated to `DomainConfiguration` implementations.

Alternatives Considered

- **Hardcode domain logic in each service**: Rejected. Couples the platform to a single domain and prevents reuse.
- **Database-driven configuration**: Deferred. Adds infrastructure dependency. SPI-based configuration is simpler and testable.
- **Remove all domain logic from platform**: Rejected. The platform must demonstrate real AI behavior. DomainConfiguration keeps domain logic while making it swappable.

Consequences

- **Extension point exists**: New domains implement `DomainConfiguration` without touching platform code
- **Migration path**: Existing hardcoded domain rules in 8 services can migrate to `DomainConfiguration` implementations incrementally
- **Testability**: Domain configurations can be tested independently

Trade-offs

- Currently a forward-looking SPI — the 8 existing services still embed their configurations rather than consuming `DomainConfiguration`
- No multi-domain routing (which `DomainConfiguration` to use for a given query)
- No domain discovery mechanism

Future Evolution

- Migrate 8 services to consume `DomainConfiguration`
- Multi-domain routing based on query classification
- Domain configuration discovery (`List<DomainConfiguration>` injection for multi-domain setups)
- External domain configuration via YAML/JSON files

See also: [[ADR-003]], [[ADR-007]]

ADR-018-provenance-graph

Status

Accepted. Implemented in `platform-neo4j/src/main/java/com/cognitera/platform/neo4j/model/GraphNode.NodeProvenance`.

Context

Knowledge graphs generated from AI extraction are only trustworthy if every node and edge can be traced back to its source. Without provenance, a graph node claiming "Acme Corporation is an ORGANIZATION" cannot be audited — was this extracted from a financial report or hallucinated by an LLM?

Decision

Every graph node and relationship carries `NodeProvenance`:

Field	Purpose
`sourceDocumentId`	Which document was the source
`chunkId`	Which chunk within the document
`chunkOffset`	Position within the chunk
`extractionConfidence`	How confident was the extraction (0.0–1.0)
`extractionTimestamp`	When was it extracted
`extractionModel`	Which model performed the extraction
`promptVersion`	Which prompt template version was used
`provider`	Which AI provider performed the extraction

Provenance is captured during enrichment (`DefaultEnrichmentService`), bridged through `EnrichmentHook`, and persisted to Neo4j by `GraphEnrichmentService`. The data flows: `EnrichmentContext` → `EnrichmentResult` → `GraphNode` (with `NodeProvenance`) → Neo4j.

Alternatives Considered

- **No provenance**: Rejected. An unauditable knowledge graph is useless for enterprise applications.
- **Separate provenance table in PostgreSQL**: Rejected. Graph data should carry its own provenance. A separate store creates consistency challenges.
- **Blockchain-based provenance**: Rejected. Over-engineered. Immutable audit log in PostgreSQL + provenance in Neo4j provides sufficient traceability.

Consequences

- **Full traceability**: Every graph element can be traced to its source document, chunk, model, prompt, and provider
- **Confidence-aware**: Extraction confidence enables filtering low-confidence extractions
- **Reproducibility**: Re-extraction with different models/prompts can be compared by timestamp and version

Trade-offs

- Provenance adds storage overhead to every node and relationship

- `extractionConfidence` is self-reported by the extraction method (not independently verified)
- Provenance is not yet leveraged for retrieval scoring (e.g., boost high-confidence extractions)

Future Evolution

- Confidence-weighted retrieval: boost documents with high-confidence extractions
- Provenance visualization in the UI
- Automated re-extraction when prompt version changes
- Provenance chain: "entity A was extracted from chunk B of document C by model D using prompt E at time F"

See also: [[ADR-007]], [[ADR-008]], [[ADR-009]]

ADR-019-testing-strategy

Status

Accepted. Implemented across `platform-ai/src/test/` and `platform-api/src/test/`. Documented in `TESTING.md`.

Context

An Enterprise AI Platform requires confidence that every architectural subsystem behaves correctly. Tests must validate behavior, not implementation details. A reader should understand the platform architecture by reading the tests.

Decision

Adopt a **5-layer testing strategy**:

Layer	Location	Execution	Purpose
Unit	`platform-ai/src/test/ … /unit/`	`mvn test`	Validate components in isolation
Integration	`platform-api/src/test/ … /ai/`	`mvn verify`	Validate subsystems with Spring context
Architecture	`platform-api/src/test/ … /architecture/`	`mvn verify`	Validate end-to-end behavior
Contract	`platform-ai/src/test/ … /contract/`	`mvn verify`	Validate SPI stability
Playwright (UI)	`e2e-tests/playwright/`	`mvn verify -Pui-tests`	Validate browser behavior

Key principles:

- **Behavior over implementation**: Tests verify what the platform does, not how it does it

- **Executable documentation**: Test names describe behavior (`"routes openai:gpt-4o to openai provider"`)
- **Realistic scenarios**: Architecture tests use a `test-corpus/` with real document types and expected behaviors
- **Contract tests**: Every `ChatCompletionProvider` implementation must satisfy the abstract `ChatCompletionProviderContract`
- **No mocks for platform internals**: Unit tests mock external providers; integration tests use real Spring context with H2

157 tests validate: Prompt Registry (12), Model Capability Registry (12), Provider Router (14), Provider Contracts (5), Retrieval Orchestrator (3), Evaluation Engine (8), Workflow Engine (10), Semantic Enrichment (14), Graceful Degradation (9), GraphRAG (4), AI Pipeline (10), Browser UI (12), Platform (91).

Alternatives Considered

- **Coverage-driven testing**: Rejected. Coverage percentage does not measure architectural confidence. 157 behavioral tests provide more value than 500 getter tests.
- **BDD framework (Cucumber)**: Rejected. Adds framework complexity. JUnit 5 `@DisplayName` + `@Nested` provides sufficient behavior description.
- **No UI tests**: Rejected. Browser tests validate user-visible behavior that unit tests cannot.

Consequences

- **Architecture as tests**: Test structure mirrors platform architecture
- **Confidence, not coverage**: Every major subsystem has executable proof of correctness
- **CI-ready**: `mvn clean verify` runs all tests except Playwright (separate profile)
- **Contract protection**: New providers must satisfy SPI contracts

Trade-offs

- Playwright tests require a running application (separate Maven profile)
- `test-corpus/` is small (3 documents) — sufficient for architectural validation, not exhaustive
- No performance regression tests yet

Future Evolution

- Performance smoke test suite (`mvn verify -Pperformance`)
- Extended test corpus with multilingual documents
- Snapshot tests for prompt rendering output
- Automated evaluation regression: "did this prompt change degrade faithfulness?"

See also: [[ADR-020]], TESTING.md

ADR-020-platform-philosophy

Status

Accepted. Embodied in every architectural decision across the platform.

Context

The platform is not a chatbot. It is not a RAG demo. It is a reference implementation of an Enterprise AI Platform designed to be reusable across multiple AI-powered applications. Every architectural decision flows from this philosophy.

Decision

The platform is governed by these design principles:

1. AI is Infrastructure

AI is not a feature. It is infrastructure, like databases or message queues. Business logic depends on abstract interfaces (`ChatCompletionProvider`, `EmbeddingProvider`), not concrete implementations (`OllamaChatProvider`, `OpenAiChatProvider`). Adding a new AI provider requires zero changes to business logic.

2. Modular Monolith Over Microservices

Module boundaries are enforced at compile time. Clear dependency direction prevents cycles. The architecture supports future extraction into microservices but does not prematurely distribute.

3. Graceful Degradation is Mandatory

Every external dependency is optional. The platform starts with zero infrastructure and gains capabilities as services become available. Keyword search works without Qdrant. Retrieval works without Neo4j. Inference works without Ollama.

4. Explainability by Default

Every AI operation produces auditable metadata: which model, which prompt, which strategy, which documents, how long it took. Explainability is not a feature toggle — it is built into the orchestration layer.

5. Documents Are the Knowledge Source

Knowledge is extracted from documents automatically during ingestion. There is no manual knowledge base, no CRUD interface for entities. The enrichment engine extracts entities, concepts, and relationships. The knowledge graph is auto-generated.

6. Domain Independence

The platform core contains no domain-specific logic. Domain customization happens through `DomainConfiguration` implementations. The platform can serve contract intelligence, financial analysis, compliance, or technical documentation from the same codebase.

7. Testing as Architecture Documentation

Tests validate behavior, not implementation. Test structure mirrors platform architecture. A Principal Engineer should understand the platform by reading the test suite.

8. Production Readiness

Every subsystem considers: how does it fail? How is it observed? How is it configured? Conditional beans, health indicators, metrics, structured logging, and configuration properties are not afterthoughts — they are first-class concerns.

Alternatives Considered

The alternative philosophies considered and rejected:

- **"Move fast and break things"**: Incompatible with enterprise AI. Every decision is deliberate, documented, and testable.
- **"Maximize features, minimize architecture"**: Would produce a prototype, not a platform. The architecture is the product.
- **"AI is magic"**: Treating AI as opaque magic leads to unmaintainable systems. Every AI operation is instrumented, evaluated, and auditable.

Consequences

- **Reusable platform**: Future applications (contract intelligence, financial analysis, compliance) can be built on the same foundation
- **Clear extension points**: `DomainConfiguration`, `ChatCompletionProvider`, `GraphSearchProvider` define where the platform grows
- **Professional engineering**: The codebase demonstrates architecture, testing, observability, and resilience — not just AI integration
- **9 modules, 157 tests, 0 failures**: Every architectural subsystem participates in real execution paths

Trade-offs

- Architectural rigor means more interfaces and abstractions than a prototype would have
- The platform is not optimized for the shortest path to a demo — it is optimized for long-term maintainability
- Some SPIs (`DomainConfiguration`) are forward-looking and not yet consumed by all eligible services

Future Evolution

This document should remain stable. New ADRs should reference these principles. If a proposed change conflicts with a principle, it should either be rejected or this ADR should be amended with explicit rationale.

See also: [[ADR-001]], [[ADR-003]], [[ADR-012]], [[ADR-016]], [[ADR-017]], [[ADR-019]]
